面向工人柔性的柔性作业车间调度：工人感知关键路径禁忌搜索

匿名作者

2026 年 4 月 28 日

摘要

经典柔性作业车间调度问题（FJSSP）主要描述机器柔性，却通常忽略工人资源。在真实人机协同车间中，一道工序只有同时选择兼容机器和兼容工人才可加工，并且加工时间由工序、机器、工人三元组共同决定。带工人柔性的 FJSSP（FJSSP-W）或双资源约束 FJSP（DRCFJSP）因此引入第三类决策变量和第三类资源冲突边。这意味着局部搜索不能只在机器析取图上移动工序，再事后修复工人冲突；析取图、邻域动作、近似评价和合法性判断都必须显式感知工人资源。本文给出一套面向比赛场景的工人感知关键路径禁忌搜索方法。该方法继承强单解 FJSP 局部搜索的轨迹搜索思想，但将表示、析取图、邻域、合法性检查和提交级结果生成流程全面扩展到机器与工人双资源。在当前 30 个 Scenario 1 官方算例上，最新审计通过的结果相对于参考提交在 11 个算例上改进最优 makespan，在其余 19 个算例上保持最优值不退化；30 个算例的最优值总和从 38496 降至 38425，十次运行平均值总和从 39549.3 降至 38464.3。

关键词： 柔性作业车间调度；工人柔性；双资源调度；禁忌搜索；局部搜索；热启动

1 引言

柔性作业车间调度问题（FJSSP）是制造系统调度中的经典 NP-hard 问题。在该问题中，每道工序可以从若干候选机器中选择一台进行加工，目标通常是最小化最大完工时间。这一抽象对机器柔性建模很有效，但对真实人机协同车间仍然不够。在实际生产中，机器往往需要由具备相应技能的工人操作；同一台机器在不同工人操作下也可能具有不同加工时间。如果忽略工人资源，得到的排程可能只是在机器维度上可行，在真实车间中仍然会因为工人冲突而不可执行。

FJSSP-W 进一步引入工人柔性：每道工序不仅需要选择机器，还需要选择兼容工人；加工时间不再只依赖机器，而是依赖工序、机器和工人的组合 [1–5]。在 DRCFJSP 文献中，这一问题也被描述为机器分配、工人分配和工序排序三个子问题的联合优化。这个扩展看起来只是多了一类资源，但对局部搜索的影响很大。标准 FJSSP 的主要冲突来自工件顺序和机器占用，而 FJSSP-W 同时存在工人占用冲突。如果搜索仍然只看

机器边，那么一个表面上缩短加工时间的移动，可能会把工序插入拥堵的工人队列，最终使真实 makespan 变差，甚至产生不可行排程。

近年不少研究进一步加入绿色指标、工人学习、疲劳、分布式工厂等扩展约束，使问题越来越贴近复杂制造场景。相比之下，FJSSP-W 比赛保留的是最核心的双资源可行性和 makespan 目标。这使它很适合验证一个基础问题：标准 FJSP 中强大的关键路径局部搜索，是否可以升级为真正的双资源局部搜索。

本文的算法受 Zhang 等人强单解 FJSP 局部搜索思想启发 [6]，但本文叙事重点不是“修改某个已有算法”，而是给出 worker-aware 的升级方式。我们的目标不是重新设计一个种群式元启发式，而是把机器感知的局部搜索变成同时感知机器和工人的轨迹搜索。

本文贡献可以概括为四点：

- 给出面向 FJSSP-W 的三元解表示，将工序序列、机器分配和工人分配同时纳入搜索状态；
- 在析取图中加入工人边，使关键结构、合法性判断和近似评价都能看到工人资源；
- 设计包含顺序移动、机器重分配、工人重分配和机器工人联合重分配的混合邻域；
- 建立热启动探索、十种子提交级再生成和审计晋级流程，避免把单个幸运结果误当成合规提交。

2 问题定义

设 O 为全部工序集合。每道工序 $i \in O$ 属于某个工件，因此同一工件内部工序之间存在固定先后关系。对每道工序，允许的加工方案是若干个二元组 (m, w)，其中 m 是可行机器，w 是可行工人。加工时间记为 $p_{i,m,w}$；如果三元组不可行，则对应加工时间不存在。

一个完整排程需要为每道工序确定开始时间 s_i、机器 m_i 和工人 w_i。排程必须满足以下约束：

1. 工件内部的工序先后约束；
2. 同一机器上的工序不能时间重叠；
3. 同一工人处理的工序不能时间重叠；
4. 每道工序所选机器工人组合必须可行，即 $p_{i,m_i,w_i} > 0$。

优化目标是最小化最大完工时间：

$$C_{\max} = \max_{i \in O} \left(s_i + p_{i,m_i,w_i} \right).$$

3 算法

3.1 从机器单资源搜索到双资源搜索

经典 FJSSP 局部搜索面向标准 FJSSP。在该问题中，一个移动可以抽象为“把工序 i 放到机器 m 的位置 k”。在 AWLS 这一类强单解局部搜索中，算法维护单个当前解，围绕关键路径生成候选移动，用禁忌表避免短期循环，并用快速 makespan 估计来筛选邻域。这一类方法最有辨识度的机制之一是自适应权重：每个工序维护累计权重和空闲计数，近似评价时把权重转换为额外加工时间。当某次移动未能改进当前解时，被移动工序在之后的估计中会变得“不那么有吸引力”；当出现新的全局最好解时，权重被重置。因此，禁忌表提供短期记忆，自适应权重提供长期扰动，两者共同帮助搜索跳出局部最优。

FJSSP-W 使得原版移动定义不再充分。一个移动不仅要说明工序放在哪台机器上，还要说明由哪个工人加工，以及它在对应工人队列中的相对位置。因此，我们保留单解禁忌搜索框架，但把搜索状态、析取图、邻域、近似评分和精确验证全部扩展到工人维度。当前提交级结果使用的是保守的工人感知版本：以工人边和精确解码为合法性底线，而不是把完整自适应权重直接作为当前结果的来源。完整 AWLS 权重项仍是下一步值得重新开启和调参的消融方向。

3.2 解表示与解码

算法用三部分表示一个解：

$$x = (\pi, M, W),$$

其中 π 是工件序列表示，M 给出每道工序的机器选择，W 给出每道工序的工人选择。如果工件 j 在 π 中第 k 次出现，则该位置代表工件 j 的第 k 道工序。这样，工件内部先后关系由出现次数隐式保证，而 M_i 和 W_i 决定工序 i 的具体加工组合。

解码器从左到右扫描 π。当遇到工件 j 的下一道工序 i 时，开始时间取三个 ready time 的最大值：

$$s_i = \max\{R_j^{job},\ R_{M_i}^{mach},\ R_{W_i}^{work}\}.$$

其中 R_j^{job} 是工件 j 上一道工序的完工时间，$R_{M_i}^{mach}$ 是所选机器当前可用时间，$R_{W_i}^{work}$ 是所选工人当前可用时间。调度完工序 i 后，这三个 ready time 都用 p_{i,M_i,W_i} 更新。只要每个 (M_i, W_i) 都在可行集合中，这个左移解码器就能产生一个可行主动排程。

3.3 加入工人边的析取图

标准 FJSSP 的局部搜索通常基于析取图理解排程。图中包含工件先后边和机器析取边。在 FJSSP-W 中，我们加入第三类边：工人边。对一个已解码排程，构造如下边集合：

- 工件边：同一工件相邻工序之间的先后关系；
- 机器边：同一机器队列中相邻工序之间的先后关系；
- 工人边：同一工人队列中相邻工序之间的先后关系。

工人边不是装饰性的。它决定了工人重分配之后是否可行，也影响关键路径判断。在这个图中，关键工序是至少位于一条源点到汇点最长路径上的工序。由于最长路径现在可能通过机器边，也可能通过工人边，所以即使机器队列完全不变，一个纯工人移动也可能改变关键结构。

3.4 精确合法性验证

所有被接受的移动都必须经过精确验证。验证包括三步：

1. 检查被修改工序的机器工人组合是否在预计算可行列表中；
2. 用同时包含工件、机器和工人 ready time 的解码器重新计算排程；
3. 根据解码结果重建机器队列和工人队列，检查是否存在重叠。

第三步看似冗余，但对比赛代码非常重要。它可以捕捉近似评价或增量更新中的实现错误，防止不可行排程进入最终 CSV。

3.5 邻域设计

搜索使用混合邻域。顺序邻域包括交换、插入、相邻交换、块交换、块迁移、双移动和反转。分配邻域包括机器重分配、工人重分配、机器相邻交换、工人相邻交换，以及机器工人联合重分配。所有分配移动都先经过工序级可行列表过滤，不可行的机器工人组合不会进入精确评价。

邻域采样偏向当前关键结构。对顺序移动，算法优先选择位于长机器队列或长工人队列附近的工序，因为这些队列更可能决定 $C_{\max}$。对分配移动，算法从该工序的可行组合中选择候选，并优先考虑加工时间较短或目标队列局部压力较小的组合。与标准 FJSSP 最大的区别是：重分配不再只是机器决策。如果只改变 M_i 而不重新考虑 W_i，瓶颈可能只是从机器队列转移到工人队列。

从概念上说，原版 AWLS 的移动 (i, m, k) 被扩展为两类移动。资源移动改变 (M_i, W_i)；顺序移动在机器队列、工人队列或二者共同诱导的局部结构中改变工序顺序。对联合分配移动，算法先选择可行组合 $(m, w) \in E_i$，再测试由当前机器队列和工人队列决定的插入位置。这样可以减少大量明显不可能的三元组合，使有限的精确评价预算集中在更可能可行的候选上。

3.6 禁忌搜索与近似筛选

主搜索是带 aspiration 的禁忌搜索。每类移动都有相应的禁忌键，用来记录被改变的顺序或分配属性。每次迭代中，算法采样一批候选移动，先用快速筛选层评分，再只对排名靠前的 K 个候选做精确解码。近似筛选只负责排序，不负责最终接受。

近似评分继承了 AWLS 中“用估计值筛选邻域”的思想，但从单一机器队列扩展为机器和工人两个资源队列。对影响工序 i 的候选移动 a，评分可以写成：

$$\hat{C}(a) = \Delta p_i(a) + \rho_m(a) + \rho_w(a) + \lambda Z_i.$$

其中 $\Delta p_i(a)$ 是加工时间变化，$\rho_m(a)$ 是目标机器局部队列压力，$\rho_w(a)$ 是目标工人局部队列压力，Z_i 是由 AWLS 工序权重和空闲计数诱导的额外加工时间。当前保守版本取 $\lambda = 0$，即不把完整 AWLS 权重项计入当前提交级结果。保留该公式的原因是说明：原版 AWLS 自适应权重可以自然接入工人感知估计器，但需要单独调参和消融验证。

算法 1 给出整体流程。禁忌表记录已接受移动的反向属性，例如工序原位置或原机器工人组合。如果某个移动处于禁忌状态，只有当它满足 aspiration 条件，即能够改进当前运行的历史最优值时，才允许被接受。如果没有非禁忌改进移动，算法也会接受损失最小的合法候选，以维持轨迹移动并穿过平台区。

3.7 为什么近似评价不会破坏可行性

近似评价只用于决定“哪些候选值得精确解码”，不会直接把解写入 incumbent 或提交文件。这一点在 FJSSP-W 中比在标准 FJSSP 中更关键。一个移动可能缩短某道工序的加工时间，却把它插入拥堵的工人队列，从而使真实 makespan 变差。因此，安全边界是：近似评价负责筛选候选，精确解码负责决定候选是否真实可行以及是否值得接受。

3.8 热启动与提交晋级流程

比赛流程本身也是算法的一部分。我们把结果分成三个层级：

1. **探索扫描：** 用较短时间和少量种子在目标算例上搜索，允许内部发现单行更优解；
2. **提交级再生成：** 如果探索扫描发现严格改进，则用该解作为热启动，对该算例重新生成十个不同种子结果；
3. **晋级审计：** 只有当完整 30 算例 CSV 每个官方算例恰好包含 10 行，并且相对参考文件没有最优值退化时，才允许晋级为候选提交。

这个流程避免了一个常见陷阱：把某个幸运单次结果直接混入 CSV，然后误以为它满足了多次运行提交规范。

4 实验设置

4.1 算例与参考结果

实验使用 FJSSP-W benchmarking 环境和 Scenario 1 提交格式。候选文件与参考提交 `atpts_final_submission.csv` 比较。当前审计通过的候选为 2026-04-27 的 `best30_v18` 结果。两个文件都包含 30 个官方算例，每个算例恰好 10 行。

4.2 运行规范

提交级再生成使用十个不同随机种子：

$$7, 19, 29, 41, 53, 67, 79, 97, 131, 173.$$

探索扫描使用较小种子批次和实例级时间限制。一旦发现改进，再用提交级十种子重新生成目标算例。主要第二阶段晋级结果使用 20000 次精确评价上限、200000 次迭代上限和邻域规模 60。

4.3 评价指标

我们报告每个算例的最优 makespan、十行平均 makespan，以及可用时达到最优行的函数评价次数。比较类别如下：

- **严格改进：** 候选最优 makespan 小于参考最优 makespan；
- **同最优但平均值更好：** 最优值相同，但十次平均 makespan 更小；
- **同最优且函数评价更少：** 最优值和平均值相同，但平均函数评价次数更低。

5 结果

表 1 给出 30 个 Scenario 1 算例的审计汇总。候选结果在 11 个算例上改进参考最优 makespan，在其余 19 个算例上保持最优值相同，没有任何最优值退化。

表 2 列出所有严格改进算例。最大的单算例改进主要出现在 DPpaulli 和 Hurink vdata 系列。30 个算例参考最优值总和为 38496，候选结果降至 38425，总下降 71。十次运行平均 makespan 总和从 39549.3 降至 38464.3。

5.1 第二阶段发现

两个最近的改进说明了有界探索流程的价值。首先，对 Hurink vdata 30 工人算例的有界扫描发现了 1069 的热启动解；提交级十种子再生成后进一步得到 1064。其次，对 Hurink vdata 18 工人算例的扫描发现了 1036；再生成后十行全部达到 1036，并且从该热启动出发不需要额外函数评价即可保留该值。两个改进都通过了完整审计。

表 1: 30 个 Scenario 1 算例的审计汇总。

类别	数量
严格最优值改进	11
同最优但平均 makespan 更低	14
同最优但函数评价次数更低	5
最优值退化	0
官方算例数	30
每个算例行数	10

表 2: 候选结果相对参考最优 makespan 的严格改进。

算例	参考最优	候选最优	差值	参考平均	候选平均
3_DPpaulli_15_workers	2213	2199	-14	2241.1	2208.9
2d_Hurink_vdata_30_workers	1075	1064	-11	1084.3	1068.1
3_DPpaulli_18_workers	2215	2204	-11	2242.0	2210.6
3_DPpaulli_9_workers	2087	2077	-10	2099.2	2083.5
2c_Hurink_rdata_38_workers	1539	1533	-6	1556.8	1536.6
2c_Hurink_rdata_28_workers	789	784	-5	811.9	787.0
2d_Hurink_vdata_18_workers	1041	1036	-5	1047.5	1036.0
6_Fattahi_14_workers	546	543	-3	570.6	544.0
0_BehnkeGeiger_60_workers	403	401	-2	419.6	402.0
3_DPpaulli_1_workers	2482	2480	-2	2528.0	2481.8
6_Fattahi_20_workers	1155	1153	-2	1177.5	1154.8

6 讨论

这些结果不是简单替换幸运单行得到的。每个晋级 CSV 都保持完整 30 算例形状，每个官方算例恰好 10 行，并且相对参考提交检查最优值退化。这很重要，因为很多最强改进先在探索模式中出现，随后才被转换成合规十种子结果。

方法上的核心经验是：FJSSP-W 必须显式看见工人资源。在许多算例中，工人队列和机器队列一样可能成为瓶颈。如果邻域只检查机器冲突，近似评价可能接受一些表面有利但最终被工人可行性抵消的移动。加入工人边和工人邻接信息后，搜索拥有更接近真实排程结构的局部模型。

本文仍有两个限制。第一，目前报告的是比赛工程流程，而不是完整消融实验。各类邻域、工人感知筛选项和原版 AWLS 自适应权重项的边际贡献还需要单独分离。第二，参考比较基于十行提交文件，而不是每个算例 20 到 30 次独立重复的统计实验。如果要形成正式投稿版本，这部分还需要补齐。

7 结论

本文给出了一套面向 FJSSP-W 的工人感知热启动禁忌搜索方法。该方法继承 AWLS 的单解局部搜索思想，并将搜索状态、析取图、邻域和合法性验证扩展到机器与工人双资源约束。在当前 30 个 Scenario 1 算例上，该方法改进 11 个参考最优值，另外 19 个保持最优值相同，并在没有最优值退化的前提下降低了最优值总和和平均值总和。后续工作应重新开启并调优完整 AWLS 自适应工序权重项，补充统计实验，并对工人边、分配移动、近似筛选和热启动晋级流程进行消融分析。

参考文献

[1] Guiliang Gong, Raymond Chiong, Qianwang Deng, and Xuran Gong. 2020. A Hybrid Artificial Bee Colony Algorithm for Flexible Job Shop Scheduling with Worker Flexibility. *International Journal of Production Research* 58, 14 (2020), 4406–4420. https://doi.org/10.1080/00207543.2019.1653504

[2] Xuran Gong, Qianwang Deng, Guiliang Gong, Wei Liu, and Qinghua Ren. 2018. A Memetic Algorithm for Multi-Objective Flexible Job-Shop Problem with Worker Flexibility. *International Journal of Production Research* 56, 7 (2018), 2506–2522. https://doi.org/10.1080/00207543.2017.1388933

[3] David Hutter, Thomas Steinberger, and Michael Hellwig. 2024. Benchmarking Worker Flexibility in Flexible Job Shop Scheduling. In *Proceedings of the 2024 IEEE Congress on Evolutionary Computation*. https://doi.org/10.1109/CEC60901.2024.10611934

[4] David Hutter, Thomas Steinberger, and Michael Hellwig. 2025. Comprehensive Benchmarking Environment for Worker Flexibility in Flexible Job Shop Scheduling Problems. arXiv:2501.16159 [cs.AI] https://arxiv.org/abs/2501.16159

[5] Changao Li, Leilei Meng, Saif Ullah, Peng Duan, Biao Zhang, and Hongyan Sang. 2025. Novel Hybrid Algorithm of Cooperative Evolutionary Algorithm and Constraint Programming for Dual Resource Constrained Flexible Job Shop Scheduling Problems. *Complex System Modeling and Simulation* 5, 3 (2025), 236–251. https://doi.org/10.23919/CSMS.2024.0041

[6] Junjie Zhang, Zhipeng Lü, Junwen Ding, Zhouxing Su, Xinyu Li, and Liang Gao. 2025. An Effective Local Search Algorithm for Flexible Job Shop Scheduling in Intelligent Manufacturing Systems. *Engineering* 50 (2025), 117–127. https://doi.org/10.1016/j.eng.2024.07.022 Available online 2024.

算法 1 工人感知热启动禁忌搜索

输入： 算例 I，随机种子 r，初始解或热启动解 x_0，预算 B

输出： 最优可行解 $x^\star$

```
为每道工序 i 构造可行列表 E_i = {(m, w, p_{i,m,w})}
x ← 修复并解码 (x_0, E)；若失败则随机构造可行解
x* ← x；初始化禁忌表 T ← ∅ 和可选 AWLS 权重 Z_i ← 0
while 预算 B 未耗尽 do
    围绕 x 采样顺序、机器、工人和联合分配移动
    for all 候选移动 a do
        if a 选择了不可行机器工人组合 then
            丢弃 a
        else if a 为禁忌且不满足 aspiration then
            丢弃 a
        else
            用加工时间变化、机器压力、工人压力和可选 Z_i 对 a 评分
        end if
    end for
    选取评分最好的 K 个移动进入精确验证
    对每个候选用工件、机器、工人 ready time 精确解码
    按接受准则选择最好的合法候选 y
    if 没有候选通过精确验证 then
        从 x* 重启或重新采样
    else
        x ← y；将反向移动属性加入 T；更新可选 AWLS 权重
        if C_max(x) < C_max(x*) then
            x* ← x
        end if
    end if
end while
return x*
```